

DATA STRUCTURE REPORT

TERM PROJECT – BASEBALL STAT DATABASE

학번	2022141030
이름	유성준
학과	화공생명공학과
제출일	2023년 6월 18일
교수	이상훈

목차

1. INTRODUCTION & DATA ACQUISITION
2. DATA STORAGE
 - A. PLAYER, DERIVEDPLAYER CLASS
 - B. DBLINKEDLIST & HEAP CLASS
 - C. HANDLER CLASS
3. FUNCTION REALIZATION
4. EXAMPLE CASE
5. CONCLUSION
6. ADDITIONAL UTILIZATION & CONCLUSION

INTRODUCTION & DATA ACQUISITION

이번 프로젝트의 목표는 취득한 특정 타입의 데이터에 대해 적절한 데이터 구조를 구축하고 이를 이용해 사용자가 여러 서비스를 취할 수 있도록 여러 함수를 구현하는 것이었다.

따라서 내가 가장 관심 있는 분야 중 데이터의 축적과 자료 분석이 적극적으로 활용되고 있는 야구의 기록에 주목하기로 하였다. 현대 야구에서는 경기 중의 타자와 투수들의 공격, 수비, 주루 등을 통해 발생하는 각종 기록을 데이터베이스에 저장한 뒤 이 기록들을 통계학적으로 분석하여, 어떤 선수가 가장 뛰어난 공을 던지고, 가장 좋은 타격 기술을 가지고 있는지 수치화 하는 것이 매우 중요한 부분이 되었다. 이 과정에서 막대한 양의 선수들의 데이터를 다루기 때문에 효율적이고 빠른 분석을 위해서는 정렬과 검색이 필수적이다. 따라서 나는 데이터 구조 강의에서 지금까지 배운 여러 자료 구조와 알고리즘을 이용하여 이러한 정렬 및 검색 기능을 제공해줄 수 있는 프로그램을 작성하기로 하였다.

우선 기본적으로 '이름'과 '팀'을 가진 각 플레이어에 대해 기록할 데이터는 다음과 같다:

```
7  enum STAT {
8      /*
9      [야구의 각종 스탯을 열거형으로 정리]
10     타석, 타수, 득점, 안타, 1루타, 2루타, 3루타, 4루타
11     타점, 볼넷, 사구, 고의사구, 삼진, 병살, 번트, 희생
12     */
13     PA, AB, R, H, B_1, B_2, B_3, B_4,
14     RBI, BB, HBP, IBB, K, DP, SH, SF
15 }
```

이를 열거형을 통해 나타내어 필요한 경우 데이터의 종류를 더 추가하거나 변경할 수 있도록 하였다. 위 데이터를 통해 다음과 같은 비율 스탯을 계산할 수 있다.

```
17  enum RATIO_STAT {
18      /*
19      [야구의 각종 비율 스탯을 상수로 정리]
20      타율, 출루율, 장타율
21      OPS: 출루율 + 장타율
22      OPSP(OPSP+): OPS를 통해 해당 타자의 타격 능력을 수치화 (평균 100)
23      wOBA: 타석 당 공격에 얼마나 기여했는지 나타내는 지수
24      wRAA: 타자가 리그 평균 타자에 대해 얼마나 득점에 기여했는지 나타내는 점수
25      wRC+: 해당 타자의 전반적인 타격 능력을 수치화 (평균 100, 10 증가 = 평균보다 10% 뛰어남)
26      */
27     BA, OBA, SLG, OPS, OPSP, wOBA, wRAA, wRC+
28 }
```

타율은 H/AB , 출루율은 $(H+BB+HBP)/(AB+BB+HBP+SF)$, 장타율은 $(B_1 + 2B_2 + 3B_3 + 4B_4)/(AB)$ 공식을 통해 구할 수 있다. 이 출루율과 장타율을 더하여 구하는 OPS와 OPS를 통해 계산하는 OPS+는 단순하면서도 야구 관계자는 물론 야구를 보는 일반인도 쉽게 타자의 능력을 알아볼 수 있는 매우 강력한 데이터이다.

wOBA, wRAA 등은 통계학적 분석을 통해 각 데이터에 가중치를 부여하여 각 타석당 타자의 기여도

를 리그 평균 기록과 연관하여 계산한 수치이다. 이를 기반으로 한 **wRC+**는 타자의 능력을 가장 정확하게 나타낼 수 있는 매우 핵심적인 수치로, 이 프로젝트에서는 데이터 구조를 활용해 wRC+를 효율적으로 계산하고, 이에 따라 정렬하고 검색하는 기능을 사용자에게 제공하는 것을 목표로 하였다.

데이터의 구성을 완료한 뒤, 한국 야구의 기록을 제공해주는 스탯티즈(statiz.co.kr)와 미국 야구의 기록을 제공해주는 팬그래프(fangraph.com), 두 사이트를 이용하여 2022년 시즌에서 뛰어난 활약을 보인 상위 30명/상위 15명 타자들의 기록을 다음과 같이 KBO.txt, MLB.txt에 저장하였다.

Junghoo Lee	Heroes	627	553	085	193	127	36	10	23	113	66	05	12	032	10	00	03
Jose Pirela	Lions	630	561	102	192	127	33	04	28	109	55	12	12	081	19	00	02
Sung-bum Na	Tigers	649	563	092	180	118	39	02	21	097	64	17	06	137	07	00	05
Ji-hwan Oh	Twins	569	494	075	133	088	16	04	25	087	62	07	02	107	11	03	03
Jeong Choi	Landers	505	414	080	110	063	21	00	26	087	66	19	03	096	06	00	06
Mike Tauchman	Eagles	648	575	088	166	113	37	04	12	043	64	07	05	104	12	00	02
Ohtani Shohei	Angels	666	586	090	160	090	30	06	34	095	072	14	05	00	00	03	00
Matt Olson	Braves	699	616	086	148	070	44	00	34	103	075	06	04	00	00	04	00
Corey Seager	Rangers	663	593	091	145	087	24	01	33	083	058	07	07	00	00	05	00
A. Santander	Orioles	647	574	078	138	081	24	00	33	089	055	01	13	00	00	05	00
Manny Machado	Padres	644	578	100	172	102	37	01	32	102	063	10	01	00	00	02	00
V.Guerrero Jr.	B.Jays	706	638	090	175	108	35	00	32	097	058	06	06	00	00	04	00

위 데이터를 통하여, 최종적으로 다음 기능을 구현하는 것을 최종 목표로 결정하였다.

1. txt 파일에 저장된 선수의 이름/팀/기록을 읽어 데이터 구조에 저장
2. 데이터 구조에 저장된 스탯을 이용해 각종 개인 기록 및 리그 평균 기록을 계산
3. 각 기록에 따라 오름차순/내림차순으로 정렬
4. 이름, 팀, 데이터 범위 등 특정 조건을 이용해 검색
5. 특정 플레이어의 삭제 및 스탯 변경 기능

DATA STORAGE – PLAYER.H (DERIVEDPLAYER)

데이터를 읽어오기 전에, 저장/검색/정렬/변경/삭제 등 모든 동작을 적절하게 구현할 수 있는 자료 구조가 무엇인지 생각해보았다. 강의에서 배운 자료 구조는 연결 리스트, 스택, 큐, 트리, 힙 등이 있는데, 이 중 스택, 큐, 트리 등은 처음/끝이 아닌 데이터를 지우기 까다로운 점을 고려하여 제외하였다. 따라서 연결 리스트 중 이중 연결 리스트를 이용하여 데이터를 기본적으로 저장하고, 데이터에 따른 정렬은 또다른 자료 구조인 힙을 활용하고자 하였다.

플레이어의 기본적인 데이터를 저장하는 동시에 연결 리스트의 노드가 되는 Player 클래스는 다음과 같이 구성하였다.

```
33 class Player { // 선수의 정보를 관리하는 클래스
34     protected:
35         // 이름, 팀, 스탯 수치를 저장할 배열
36         string name;
37         string team;
38         int stats[STAT_SIZE];
39         double ratio_stats[RATIO_STAT_SIZE];
40
41         // 이중 연결 리스트에서 다음 노드, 이전 노드
42         Player* next;
43         Player* prev;
44
45         // 기본 스탯을 기반으로 비율 스탯을 계산
46         void calculate_ratio(int[STAT_SIZE], double[RATIO_STAT_SIZE]);
47
48     public:
49         // Player 클래스의 생성자와 소멸자
50         Player(string, string, int[STAT_SIZE]);
51         virtual ~Player(void);
52
53         // Player의 정보를 반환
54         string getName(void);
55         string getTeam(void);
56         int* getStats(void);
57         double* getRatio(void);
58
59         // 이중 연결 리스트 링크 getter/setter
60         Player* getNext(void);
61         Player* getPrev(void);
62         void setNext(Player*);
63         void setPrev(Player*);
64
65         void print_stats(void); // Player의 스탯 출력
66         virtual void print_league(int) = 0; // 해당 Player가 속한 리그의 평균 스탯 출력
67         virtual void calculate_lg_ratio(void) = 0; // 리그 평균 스탯 기반 플레이어 비율 스탯 계산
68     };
--
```

name, team, stats, ratio_stats는 플레이어의 정보를 저장하는 멤버 변수이고, Player* 형의 next, rprev는 이중 연결 리스트에서 이전 노드와 다음 노드를 잇는 링크이다. calculate_ratio 함수는 stats를 바탕으로 ratio_stats를 계산하는 멤버 함수이다. 위 멤버는 Player 클래스를 상속받은 클래스에서 자유롭게 사용할 수 있도록 protected 접근자로 지정되어 있다.

public 멤버 함수로는 플레이어의 정보와 연결 리스트의 링크를 관리하는 get/set 함수 시리즈, 클래스의 생성자와 소멸자, 그리고 해당 플레이어 노드의 스탯을 출력하는 print_stats 함수 등으

로 구성되어 있다. 이 중 소멸자는 가상 함수, print_league나 print_lg_ratio 함수는 순수 가상 함수로 되어 있는데, 이는 Player 클래스를 상속받아 각 소속 리그에 대한 정보를 기록할 KBOPlayer, MLBPlayer 클래스에 공통적으로 존재할 함수들이기 때문이다.

```

3      // Player 클래스의 생성자
4      Player::Player(string _name, string _team, int _stats[STAT_SIZE])
5      : name(_name), team(_team) {
6
7          // 이름, 팀, 스탯, 링크 초기화
8          for (int i = 0; i < STAT_SIZE; i++)
9              stats[i] = _stats[i];
10         next = prev = NULL;
11
12         // 기본 스탯 기반 비율 스탯 계산
13         calculate_ratio(stats, ratio_stats);
14     }

```

Player 클래스의 멤버 함수들의 구현은 매우 간단하다. 생성자의 경우 위와 같이 이름, 팀, 스탯 데이터에 대하여 매개변수를 각각 대입해준 뒤, next, prev를 NULL로 지정해주고 stats를 기반으로 ratio_stats를 계산하도록 하였다. 나머지 getter/setter 함수들은 각 멤버를 반환하거나 매개변수를 대입해주는 방식으로 구현하였다.

```

19      // 기본 스탯을 기반으로 비율 스탯(타율, 출루율, 장타율, OPS, wOBA)을 계산
20      void Player::calculate_ratio(int _stats[STAT_SIZE], double _ratio[RATIO_STAT_SIZE]) {
21          _ratio[BA] = ((double)_stats[H] / (double)_stats[AB]);
22          _ratio[OBA] = ((double)(_stats[H] + _stats[BB] + _stats[HBP]) / ((double)(_stats[AB] + _stats[BB] + _stats[HBP] + _stats[SF])));
23          _ratio[SLG] = ((double)(_stats[B_1] + _stats[B_2] * 2 + _stats[B_3] * 3 + _stats[B_4] * 4) / ((double)(_stats[AB])));
24          _ratio[OPS] = _ratio[OBA] + _ratio[SLG];
25          _ratio[wOBA] = ((double)(0.689 * (double)(_stats[BB] - _stats[IBB]) + 0.720 * (double)_stats[HBP] + 0.884 * (double)_stats[B_1]
26                          + 1.262 * (double)_stats[B_2] + 1.601 * (double)_stats[B_3] + 2.072 * (double)_stats[B_4]) /
27                          ((double)(_stats[AB] + _stats[BB] - _stats[IBB] + _stats[HBP] + _stats[SF])));
28      }

```

calculate_ratio 함수는 가독성이 떨어지지만 타율, 출루율, 장타율, OPS, wOBA에 대해 공식에 따라 계산한 뒤 ratio_stats에 저장한 것으로 매우 간단하다.

```

84      class KBOPlayer : public Player {
85      private:
86          static int nKBOPlayer;
87          static int KBO_stats[STAT_SIZE];
88          static double KBO_ratio_stats[RATIO_STAT_SIZE];
89          double park_factor;
90
91      public:
92          KBOPlayer(string, string, int[STAT_SIZE]);
93          ~KBOPlayer();
94          void print_league(int);
95          void calculate_lg_ratio(void);
96      };

```

Player 클래스를 public으로 상속한 KBOPlayer와 MLBPlayer 클래스의 구성은 위와 같다(MLB의 경우 위 클래스에서 park_factor 변수를 삭제한 것과 동일하다). 현재 생성된 각 소속 리그 별 플레이어의 수, 각 플레이어의 총합 스탯 및 비율 스탯을 저장하는 static 멤버 변수들이 있으며, 이는

초기에 모두 0으로 초기화된다. 멤버 함수로는 생성자와 소멸자, static 멤버를 통해 각 리그의 평균 기록을 출력하는 print_league, 리그 평균 스탯을 기반으로 OPS+, wRAA, wRC+를 계산하는 calculate_lg_ratio 함수가 있다. KBO의 경우 park_factor라는 변수가 존재하는데, 이는 각 플레이어가 소속된 팀이 홈 구장에 따라 달라지는 변수로 각 구장이 투수 친화/타자 친화인가에 따라 기록에 가중치를 부여한다.

print_league나 calculate_lg_ratio 함수는 부모 클래스의 순수 가상 함수를 재정의하는 형태이다. 이 구조를 택한 이유는 KBOPlayer나 MLBPlayer 데이터에 대한 처리를 Player형 포인터를 통해 한번에 처리할 것이기 때문이다.

```

10     // KBO Player 클래스의 생성자
11     KBOPlayer::KBOPlayer(string _name, string _team, int _stats[STAT_SIZE])
12     : Player(_name, _team, _stats) {
13         // 이름, 팀, 스탯 초기화 및 정적 변수/배열 증가
14         for (int i = 0; i < STAT_SIZE; i++)
15             KBO_stats[i] += stats[i];
16         nKBOPlayer++;
17
18         // 리그 총합 스탯에 대해 리그 평균 비율 스탯 계산
19         calculate_ratio(KBO_stats, KBO_ratio_stats);
20
21         // Player의 팀에 따라 파크 팩터 설정
22         if (team == "Landers")    park_factor = 1.052;
23         else if (team == "Heroes") park_factor = 0.957;
24         else if (team == "Wiz")   park_factor = 1.025;
25         else if (team == "Tigers") park_factor = 0.991;
26         else if (team == "Dinos") park_factor = 1.000;
27         else if (team == "Lions") park_factor = 1.052;
28         else if (team == "Giants") park_factor = 1.040;
29         else if (team == "Eagles") park_factor = 0.983;
30         else                      park_factor = 0.943;
31     }

```

상속 클래스의 생성자는 위와 같이 부모 클래스의 생성자를 이용하여 기본 스탯을 저장한 뒤 기본 스탯을 통하여 리그 총합 스탯을 업데이트하는 방식으로 구현하였다. 또한 KBOPlayer의 경우 팀에 따라서 park_factor 수치를 지정해주었다.

```

33     // KBO 클래스의 소멸자
34     KBOPlayer::~~KBOPlayer() {
35         // 정적 변수/배열 감소 후 비율 스탯 재계산
36         nKBOPlayer--;
37         for (int i = 0; i < STAT_SIZE; i++)
38             KBO_stats[i] -= stats[i];
39         if (nKBOPlayer > 0)
40             calculate_ratio(KBO_stats, KBO_ratio_stats);
41     }

```

소멸자의 경우 생성자와 유사하게 리그 총합 스탯을 업데이트하고 비율 스탯을 재계산하도록 구현하였다. 데이터 구조에서 각 데이터를 Player* 형으로 다루기 때문에 해당 소멸자가 실행되기 위해서는 Player 클래스의 소멸자를 가상 소멸자로 선언해주어야 했다.

```

43 // format에 맞추어 KBO 리그의 평균 비율 스탯 출력
44 void KBOPlayer::print_league(int count) {
45     cout.precision(3); cout << fixed;
46     cout << "KBO League (상위 " << count << "명)" << "BA" << KBO_ratio_stats[BA] << "OB" << KBO_ratio_stats[OBA]
47         << "SL" << KBO_ratio_stats[SLG] << "OPS" << KBO_ratio_stats[OPS] << endl << endl;
48 }
49
50 // 리그 평균 스탯 기반 플레이어 비율 스탯 계산
51 void KBOPlayer::calculate_le_ratio(void) {
52     ratio_stats[OPSP] = 100 * ((ratio_stats[OBA] / KBO_ratio_stats[OBA]) + (ratio_stats[SLG] / KBO_ratio_stats[SLG]) - 1) / park_factor;
53     ratio_stats[wRAA] = (ratio_stats[wOBA] - KBO_ratio_stats[wOBA]) * stats[PA] / 1.251;
54     double leaguePPA = (double)KBO_stats[R] / KBO_stats[PA];
55     ratio_stats[wRCP] = 100 * (((ratio_stats[wRAA] / (double)stats[PA]) + (leaguePPA - (park_factor * leaguePPA))) / leaguePPA + 1);
56 }

```

print_league 함수는 간단하게 KBO_ratio_stats에 기록된 데이터를 형식에 맞추어 출력하였고, calculate_le_ratio는 calculate_ratio 함수와 동일하게 OPS+, wRAA, wRC+의 공식에 따라 계산한 뒤 저장하도록 하였다. park_factor의 유무에 따라 계산식이 크게 달라지기 때문에 KBOPlayer와 MLBPlayer 클래스를 상속을 통해 분리하였다.

DATA STORAGE – DBLINKEDLIST.H

각 Player의 데이터를 저장할 이중 연결 리스트인 DBLinkedList 클래스의 구성은 다음과 같다.

```

6 class DBLinkedList { // Player의 정보를 저장하는 이중 연결 리스트
7 private:
8     Player* head; // 이중 연결 리스트의 처음
9     Player* tail; // 이중 연결 리스트의 끝
10    int nPlayer; // 연결 리스트에 저장된 노드의 수
11
12 public:
13     DBLinkedList(void); // DBLinkedList 클래스의 생성자
14     ~DBLinkedList(void); // DBLinkedList 클래스의 소멸자
15
16     int getnPlayer(void); // 리스트에 저장된 노드 수 반환
17     double getHeadStat(int stat); // 리스트의 처음/끝의 특정 스탯 반환
18     double getTailStat(int stat);
19     bool isEmpty(void); // 리스트 비어있는지 여부 확인
20     void insert_back(Player*); // 리스트의 가장 뒤에 노드 삽입
21     Player* delete_front(bool _no_return = false); // 리스트의 가장 앞 노드 삭제
22     void delete_node(Player*&); // 리스트의 한 노드 삭제
23     bool search_list(Player*&, string _team, string _name); // 리스트에서 Player 검색
24     void calculate_av(void); // 리스트의 Player 스탯 초기화
25     void print_out(int, int, string, string, double, double, bool); // 리스트 내 노드 출력
26 };

```

이중 연결 리스트의 처음과 끝을 나타내는 head와 tail, 그리고 현재 노드에 저장된 플레이어의 수를 나타내는 nPlayer를 private 멤버 변수로 하였다. 멤버 함수로는 생성자와 소멸자, 각각 nPlayer, head의 특정 스탯, tail의 특정 스탯을 반환하는 get 함수, 리스트가 비어 있는지 여부를 판단하는 isEmpty 함수, 각 플레이어의 스탯을 초기화하는 calculate_av 함수가 있다.

또한 리스트를 직접 조작하는 함수들로는 가장 앞 노드를 삭제하는 delete_front, 리스트의 특정 플레이어를 삭제하는 delete_node, 특정 노드를 검색하는 search_list, 리스트 내 플레이어의 정보를 조건에 맞추어 출력하는 print_out 함수가 있다.

생성자나 소멸자는 기존의 연결 리스트의 구현과 동일하다. 생성자는 head와 tail을 NULL로, nPlayer를 0명으로 초기화하며, 소멸자는 리스트를 순회하면서 모든 노드를 차례대로 삭제한다.

isEmpty 함수는 head가 NULL인지 여부를, getnPlayer 함수는 nPlayer 멤버를 반환하며, getHeadStat, getTailStat 함수는 head와 tail이 NULL인지 여부를 먼저 확인한 뒤 각 플레이어의 특정 스탯을 반환한다. calculate_av 함수는 리스트를 순회하면서 각 플레이어의 calculate_lg_ratio 함수를 실행시킨다.

```

41 // 리스트의 가장 뒤에 노드 삽입
42 void DBLinkedList::insert_back(Player* node) {
43     nPlayer++;
44     if (isEmpty()) {
45         head = node;
46         tail = node;
47     }
48     else {
49         Player* tmp = head;
50         while (tmp->getNext() != NULL)
51             tmp = tmp->getNext();
52         tmp->setNext(node);
53         node->setPrev(tmp);
54         tail = node;
55     }
56 }

```

// 리스트에 저장된 노드 수 증가
// 리스트가 비었을 경우
// 처음과 끝 모두 node로 지정
// 리스트에 노드가 존재할 경우
// head 부터 시작하여 tail까지 이동
// 기존 tail의 다음 노드를 node로 지정
// node의 이전 노드를 tmp로 지정
// tail을 node로 지정

Player 노드를 가장 뒤에 삽입하는 insert_back 함수의 구현은 위와 같다. 만약 리스트가 비었을 경우 해당 노드를 head이자 tail로 지정한다. 리스트에 노드가 존재할 경우 가장 끝 노드로 이동한 뒤 마지막 노드의 다음 노드를 node, node의 이전 노드를 기존 마지막 노드로 지정하고 tail을 node로 변경한다.

```

58 // 리스트의 가장 앞 노드 삭제
59 Player* DBLinkedList::delete_front(bool _no_return) {
60     if (isEmpty())
61         return NULL;
62
63     nPlayer--;
64     if (_no_return == true) {
65         Player* tmp_node = head;
66         head = head->getNext();
67         if (head == NULL) tail = NULL;
68         delete tmp_node;
69         return NULL;
70     }
71     else {
72         Player* tmp_node = head;
73         head = head->getNext();
74         if (head == NULL) tail = NULL;
75         tmp_node->setNext(NULL);
76         tmp_node->setPrev(NULL);
77         return tmp_node;
78     }
79 }

```

// 리스트가 비었을 경우 NULL 반환
// 리스트 내 노드 수 감소
// 반환값 없을 경우
// head를 다음 노드로 이동
// 다음 노드가 비었을 경우 끝도 NULL로 설정
// 노드 삭제 후 NULL 반환
// 반환값 있을 경우
// 기존 head의 next, prev 초기화 후 반환

```

81 // 리스트의 한 노드 삭제
82 void DBLinkedList::delete_node(Player*& node) {
83     if (head == node) { // node가 head일 경우 delete_front 함수로 처리
84         delete_front(true);
85     }
86     else if (tail == node) { // node가 tail일 경우
87         nPlayer--; // 리스트 내 노드 개수 감소
88         tail = node->getPrev(); // tail을 뒤에서 두번째 노드로 설정
89         node->getPrev()->setNext(NULL); // 뒤에서 두번째 노드의 다음을 NULL로 설정
90         delete node; // 노드 삭제
91     }
92     else { // 둘 다 아닐 경우
93         nPlayer--; // 리스트 내 노드 개수 감소
94         node->getPrev()->setNext(node->getNext()); // 이전 노드와 다음 노드를 서로 이어줌
95         node->getNext()->setPrev(node->getPrev());
96         delete node; // 노드 삭제
97     }
98 }

```

노드를 삭제하는 delete_front와 delete_node 함수의 구현은 위와 같다. 만약 삭제 대상이 head인 경우 delete_front를 실행한다. 삭제 대상이 tail인 경우 tail을 기존 마지막 노드의 이전 노드로, 이전 노드의 다음 노드를 NULL로 변경한 뒤 node를 삭제한다. 만약 head도 tail도 아닐 경우 node의 이전과 다음 노드의 링크를 각각 이어준 뒤 node를 삭제한다.

delete_front 함수의 경우 우선 만약 리스트가 비었을 경우 NULL을 반환한다. 삭제 후 해당 노드를 반환할 것인지 여부(_no_return)에 따라서 구현이 달라지는데, 반환할 경우 head를 다음 노드로 옮긴 뒤 head의 링크를 모두 초기화하고 해당 노드를 반환하고, 반환하지 않을 경우 노드를 그대로 삭제하고 NULL을 반환한다. 위와 같이 따로 구현한 이유는 Player의 n(League)Player 변수를 적절히 관리하기 위함이다.

```

100 // 리스트에서 Player 검색
101 bool DBLinkedList::search_list(Player*& find, string _team, string _name) {
102     Player* tmp = head;
103     find = tmp;
104     // 이름과 팀 입력이 공백일 경우 true 반환
105     if (_name.empty() && _team.empty()) { find = tmp; return true; }
106
107     // 노드를 이동하면서 이름 공백/팀 일치, 팀 공백/이름 일치, 이름/팀 모두 일치 발견 시 true
108     while (tmp != NULL) {
109         if (((_name.empty() && _team == tmp->getTeam()) || (_name == tmp->getName() && _team.empty()))
110             || (_name == tmp->getName() && _team == tmp->getTeam())) {
111             find = tmp;
112             return true;
113         }
114         tmp = tmp->getNext();
115     }
116     // 찾지 못할 경우 false 반환
117     find = NULL;
118     return false;
119 }

```

리스트에서 특정 팀과 이름을 가진 선수를 검색하는 search_list 함수의 구현은 위와 같다. 매개변수의 경우 팀과 이름, 그리고 Player*형의 레퍼런스 find를 받는데, 만약 조건에 해당되는 노드를 찾을 경우 true를 반환하는 동시에 해당 노드를 find를 통해 반환한다.

매개변수인 팀과 이름이 모두 공백인 경우 우선 true를 반환하고, 하나라도 공백이 아닐 경우 리

스트를 순회하면서 팀 일치/이름 일치/팀과 이름 모두 일치하는 경우를 검색한다. 예를 들어 _name은 공백이고 _team은 공백이 아닐 경우 팀이 일치하는 가장 첫 노드를 find를 통해 반환하게 된다. 만약 리스트에 존재하지 않을 경우 false를 반환한다.

```

134 // 리스트 내 노드 출력 <상위 count 명, 팀 일치, 이름 일치>
135 void DBLinkedList::print_out(int mode, int count, string _team, string _name, double min, double max, bool reverse) {
136     if (isEmpty())
137         cout << "리스트가 비어 있습니다." << endl;
138     else {
139         Player* tmp = NULL;
140         if (!search_list(tmp, _team, _name)) { ... }
141
142         int _count = 0;
143         tmp->print_league(count); // 해당 리그의 평균 스탯 먼저 출력
144         cout << "이름\t팀\t타율\t출루율\t장타율\tOPS\twRC+" << endl;
145         cout << "-----" << endl;
146
147         if (!reverse) {
148             tmp = head;
149             while (tmp != NULL) {
150                 // 형식에 맞추어 상위 count 명의 선수 중 조건에 맞는 선수의 스탯 출력
151                 if (_count == count) break;
152                 if ((_name.empty() && _team.empty()) || (_name.empty() && _team == tmp->getTeam())
153                     || (_name == tmp->getName() && _team.empty()) || (_name == tmp->getName() && _team == tmp->getTeam()))
154                     && ((min <= tmp->getRatio()[mode]) && (max >= tmp->getRatio()[mode])) {
155                     tmp->print_stats();
156                 }
157                 _count++;
158                 tmp = tmp->getNext();
159             }
160
161         } else {
162             tmp = tail;
163             while (tmp != NULL) {
164                 // 형식에 맞추어 상위 count 명의 선수 중 조건에 맞는 선수의 스탯 출력
165                 if (_count == count) break;
166                 if ((_name.empty() && _team.empty()) || (_name.empty() && _team == tmp->getTeam())
167                     || (_name == tmp->getName() && _team.empty()) || (_name == tmp->getName() && _team == tmp->getTeam()))
168                     && ((min <= tmp->getRatio()[mode]) && (max >= tmp->getRatio()[mode])) {
169                     tmp->print_stats();
170                 }
171                 _count++;
172                 tmp = tmp->getPrev();
173             }
174         }
175         cout << endl;
176     }
177 }

```

마지막으로 조건에 맞추어 리스트를 순회하면서 플레이어의 정보를 출력하는 print_stats 함수의 구현은 위와 같다. print_stats는 현재 정렬한 기준(mode), 상위 몇 명의 정보를 출력할 것인지(count), 특정 팀, 특정 이름, 최소, 최대, 순서 반대 여부까지 총 7개의 조건을 매개변수로 받기 때문에 구현이 조금 복잡하다.

만약 리스트가 비어 있을 경우 에러 메시지를 반환한다. 비어 있지 않을 경우 search_list 함수를 통해 해당 이름/팀을 가진 선수가 존재하는 지 여부를 우선 검사한 뒤 존재하지 않을 경우 에러 메시지를 출력하고 함수를 종료한다.

만약 해당 이름/팀을 가진 선수가 존재할 경우 해당 소속 리그의 정보를 출력하고, 리스트를 순회하면서 팀/이름 공백 및 일치 여부와 해당 값이 최소/최대 범위 안에 드는지, 정렬 기준 상위 n명 안에 드는지 if 문을 통해 체크한 뒤 print_stats 함수를 이용해 정보를 출력한다. 만약 reverse가 false일 경우 head에서 next 링크를 따라, true일 경우 tail에서 prev 링크를 따라 출력한다.

DATA STORAGE – HEAP.H

삭제/검색/조건에 맞는 출력 기능은 리스트를 통하여 구현하였는데, 해당 기능을 적절하게 사용하기 위해서는 리스트 내 데이터들의 정렬이 필요하다. 물론 연결 리스트 내에서 정렬 기능을 구현할 수 있지만, 연결 리스트 내부에서는 노드의 순서를 바꾸는 것이 까다로우며 노드의 수가 많아질수록 처리해야 할 연산의 수가 기하급수적으로 증가한다.

따라서 이번 프로젝트에서는 Max/Min Heap을 구현하여 리스트 내부의 모든 데이터를 Max/Min Heap에 push하고 다시 차례대로 pop하여 리스트에 삽입하는 방식인 힙 정렬을 사용하기로 하였다. 이 경우 데이터 제거 및 삽입 시간이 증가하겠지만 정렬을 $O(n^2)$ 에서 $O(n \log n)$ 으로 크게 줄일 수 있다. Heap의 경우 배열을 사용하는 것이 편한데, 이는 Heap은 가장 왼쪽 노드부터 차례대로 채워지는 완전 이진 트리의 형태이기 때문이며, 또한 배열을 사용할 경우 index를 통해 특정 노드에 직접 접근이 가능하기 때문이다.

```
6  class Heap {                                // 힙 정렬에 이용할 Heap 클래스
7      private:
8          Player** heap;                       // 데이터를 저장할 Heap
9          int size;                            // 현재 heap에 저장된 데이터
10         int capacity;                        // heap의 총 저장 공간
11         int mode;                           // 정렬 mode 지정 (비율 스택)
12         int order;                          // 오름차순/내림차순
13
14         void swap(Player*&, Player*&);      // Player 노드 끼리 정보 교환
15         void heapify(int);                  // Max Heap 형태로 데이터 저장
16     public:
17         Heap(int);                          // Heap 클래스의 생성자
18         ~Heap();                            // Heap 클래스의 소멸자
19         void setMode(int);                  // 정렬 mode 지정
20         void setOrder(int);
21         void resize();                      // Heap의 총 저장 공간 재설정
22         void push(Player*);                 // Heap에 Player 노드 저장
23         Player* pop();                      // Heap의 root 노드 반환
24     };
```

heap의 구성은 위와 같다. Player* 형 데이터를 저장할 Player** 형의 heap, heap에 저장된 데이터의 수 size, heap에 저장할 수 있는 총 공간 capacity, 현재 정렬 기준 mode, 오름차순/내림차순 여부를 저장하는 order가 private 멤버 변수로 지정되었다. 멤버 함수로는 생성자와 소멸자, 정렬 방식을 지정하는 setMode와 setOrder 함수, 두 데이터의 정보를 교환하는 swap 함수, Heap의 사이즈를 재지정하는 resize 함수, 데이터를 Max heap의 형태로 만드는 heapify 함수, Heap에 데이터를 삽입/제거하는 push/pop 함수가 있다.

```

3      // Heap 클래스의 생성자
4      Heap::Heap(int _capacity) {
5          capacity = _capacity;           // 총 용량 설정
6          heap = new Player * [capacity]; // Heap 동적 할당
7          size = 0;                       // Heap 내부 데이터 개수 초기화
8          order = 1;                      // 기본: 내림차순
9          mode = BA;                      // 기본: 타율
10     }
11
12     // Heap 클래스의 소멸자
13     Heap::~Heap() {
14         delete[] heap;                  // Heap 동적 할당 해제
15     }
16
17     // Player 노드 끼리 정보 교환
18     void Heap::swap(Player*& a, Player*& b) {
19         Player* tmp = a;
20         a = b;
21         b = tmp;
22     }
23

```

Heap의 기본 함수들의 구현은 매우 간단하다. capacity를 입력 받아 heap을 생성한 뒤 capacity 만큼 heap을 동적할당 해주고, size를 0으로 초기화한다. order와 mode는 각각 내림차순(-1), 타율(BA)로 초기화한다. 소멸자의 경우 동적 할당을 해제해준다. swap 함수는 call-by-reference 방식을 이용하여 두 Player* 형의 정보를 서로 교환하도록 하였다.

```

47     // 정렬 mode 지정
48     void Heap::setMode(int _mode) {
49         mode = _mode;
50     }
51
52     // 내림차순(1)/오름차순(-1) 지정
53     void Heap::setOrder(int _order) {
54         order = _order;
55     }
56
57     // Heap의 총 저장 공간 재설정
58     void Heap::resize() {
59         capacity *= 2;
60         Player** tmp = new Player * [capacity]; // 새 capacity에 맞추어 Heap 재할당
61         for (int i = 0; i < size; i++)
62             tmp[i] = heap[i];
63         delete[] heap;                      // 기존 heap 삭제 후 초기화
64         heap = tmp;
65     }

```

setMode나 setOrder 함수는 위와 같이 멤버에 매개변수를 대입하도록 구현하였다. resize 함수의 경우 현재 capacity의 사이즈를 2배 증가시킨 heap을 새로 생성한 뒤 기존의 heap에 덮어쓰우는 방법을 사용하였다.

```

24     // Max Heap 형태로 데이터 저장
25     void Heap::heapify(int index) {
26         int est = index;
27         int left = 2 * index + 1;
28         int right = 2 * index + 2;
29
30         // 왼쪽 자식 노드가 부모 노드보다 클/작을 경우 자식 노드를 est로 설정 (부모로)
31         if (left < size && compStat(heap[left], heap[est], mode) == order)
32             est = left;
33
34         // 오른쪽 자식 노드가 부모 노드보다 클/작을 경우 자식 노드를 est로 설정 (부모로)
35         if (right < size && compStat(heap[right], heap[est], mode) == order)
36             est = right;
37
38         // est의 변경이 일어났을 경우
39         if (est != index) {
40             // 자식 노드와 부모 노드 데이터 교환 후 새 자식 노드에 대해 재실행
41             swap(heap[index], heap[est]);
42             heapify(est);
43         }
44     }

```

heap의 구현에 가장 핵심이 되는 heapify 함수의 구현은 위와 같다. Max Heap을 예시로 하면 index가 부모 노드일 경우 자식 노드는 $2 * index + 1$, $2 * index + 2$ 번째 노드가 각각 왼쪽, 오른쪽 자식 노드가 되며 모두 부모 노드보다 그 값이 작아야 한다.

따라서 left, right가 현재 heap에 포함되는 size인지 검사하고 부모와 자식 노드의 데이터를 서로 비교하여 만약 Max Heap의 기준에 맞지 않으면 부모와 자식 노드를 서로 교환하고 새 자식 노드에 대해 heapify 함수를 다시 실행한다. 사용자가 설정한 조건에 따라 Max Heap, Min Heap이 구현되도록 compStat 함수와 order를 이용하여 구현하였다.

```

51     // 두 플레이어의 특정 스탯을 비교
52     int compStat(Player* a, Player* b, int mode) {
53         // a가 더 클 경우 1, b가 더 클 경우 -1, 같을 경우 0을 반환
54         if (a->getRatio()[mode] > b->getRatio()[mode])
55             return 1;
56         else if (a->getRatio()[mode] < b->getRatio()[mode])
57             return -1;
58         else
59             return 0;
60     }

```

compStat은 Player.h에 정의된 함수로 만약 a의 스탯이 b의 스탯보다 클 경우 1, 작을 경우 -1, 같을 경우 0을 반환하는 함수이다. 즉 order가 1일 경우 Max Heap, -1일 경우 Min Heap으로 각 데이터를 Heap Tree 형태로 저장할 수 있다.

```

67 // Heap에 Player 노드 저장
68 void Heap::push(Player* node) {
69     // Heap 공간의 용량이 모두 찼을 경우 resize
70     if (size == capacity)
71         resize();
72
73     // heap의 끝 공간에 node 저장, size 증가
74     heap[size] = node;
75     size++;
76
77     // index = 새로 저장된 노드, parent = 해당 노드의 부모
78     int index = size - 1;
79     int parent = (index - 1) / 2;
80
81     // 부모 노드가 새 노드보다 작을/클 경우
82     while (index > 0 && compStat(heap[parent], heap[index], mode) == (-1*order)) {
83         // 노드 교환 후 새 노드가 처음으로 부모 노드보다 작을/클 때까지 반복
84         swap(heap[index], heap[parent]);
85         index = parent;
86         parent = (index - 1) / 2;
87     }
88 }
89
90 // Heap의 root 노드 반환
91 Player* Heap::pop() {
92     if (size == 0) // Heap이 비었을 경우 NULL
93         return NULL;
94
95     Player* root = heap[0]; // Heap의 root를 반환
96     heap[0] = heap[size - 1]; // Heap의 root를 가장 마지막 노드로 지정
97     size--; // size 감소
98     heapify(0); // 다시 Max Heap 구성
99     return root;
100 }

```

Heap에 데이터를 삽입/제거하는 push와 pop 함수는 위와 같이 구현하였다. 만약 예외 상황이 발생하여 size가 capacity와 같아질 경우 heap의 크기를 증가시킨 뒤, 새 노드를 heap tree의 끝에 삽입한다. 그 후 새 노드의 부모 노드와 비교하여 만약 Max/Min Heap 기준을 만족시키지 않을 경우 부모 노드와 자식 노드를 교환한다. 새 노드가 적절한 위치를 찾을 때까지 반복하며 만약 해당 노드의 값이 최대/최소일 경우 root가 될 때까지 노드를 위로 이동시킨다.

pop 함수의 경우 heap tree의 최대/최소 값인 root를 반환하고, heap tree의 가장 끝에 있던 노드를 root로 이동시킨 뒤 heapify 함수를 통해 heap tree를 재구축한다.

DATA STORAGE – HANDLER.H

여러 데이터 구조와 함수를 구현하였으므로 이 기능들을 이용하여 사용자와 사용할 수 있는 인터페이스인 Handler 클래스를 따로 구현하기로 하였다. 이 클래스는 파일 입력을 통해 실제 데이터를 리스트에 저장하고, 리스트와 힙을 이용해 삽입/삭제/변경/검색/출력을 직접 수행하는 클래스이다. Handler 헤더는 다음과 같이 구성하였다.

```
1  #ifndef __HANDLER_H_
2  #define __HANDLER_H_
3
4  #include "DBLinkedList.h"
5  #include "Heap.h"
6  #include <fstream>
7
8  #define NO_RANGE -200.0
9  #define ALL ""
10
11 class Handler {           // 유저 인터페이스
12 private:
13     DBLinkedList list;    // 데이터 저장 연결 리스트
14     string league;        // Player가 속한 리그
15     int mode;             // 현재 정렬 중인 스탯
16     int order;            // 오름차순/내림차순 여부
17
18 public:
19     Handler(string);      // Handler의 생성자
20     void initialize(void); // 리스트 초기화
21     void sort(int _mode = BA, int _order = 1); // 힙을 이용해 리스트 정렬
22     void del_player(string team = ALL, string name = ALL); // 리스트 내 플레이어 삭제
23     void update(string _team, string _name, int, int); // 리스트 내 플레이어 스탯 업데이트
24
25     // 리스트 내 데이터 출력
26     void print_stats(int count = -1, string team = ALL, string name = ALL,
27                     double min = NO_RANGE, double max = NO_RANGE, bool reverse = false);
28     void print_stats(string team, string name = ALL);
29     void print_stats(double min, double max = NO_RANGE);
30     void print_stats(bool reverse);
31 };
32
33 #endif
```

Handler.h는 DBLinkedList.h, Heap.h, fstream 함수를 include 하여 지금까지 구현한 모든 자료 구조와 파일 입출력을 활용할 수 있도록 하였다. 또한 두 매크로 상수를 지정하였는데, NO_RANGE의 경우 최소/최대 범위를 지정하지 않았다는 의미의 상수, ALL은 특정 팀/이름 조건 없이 모든 데이터를 출력함을 의미하는 공백이다.

private 멤버로는 데이터를 처리할 DBLinkedList, 리스트에 저장될 플레이어들의 소속 리그 league, 그리고 정렬 기준인 mode와 order가 있다.

public 멤버 함수로는 생성자, list를 초기화하는 initialize, list를 정렬하는 sort, 특정 플레이어/팀을 삭제하는 del_player, 특정 플레이어의 스탯을 변경하는 update, 마지막으로 조건에 맞는 선수의 데이터를 출력하는 print_stats 함수가 있다.

```

3 // Handler의 생성자. league 문자열 초기화
4 Handler::Handler(string _league) : league(_league) { mode = BA; order = 1; }
5
6 // 리스트 초기화
7 void Handler::initialize(void) {
8     // 기존 큐에 저장되어있던 데이터 모두 초기화
9     mode = BA; order = 1;
10    int member = list.getnPlayer();
11    for (int i = 0; i < member; i++)
12        list.delete_front(true);
13
14    // KBO.txt/MLB.txt 파일 및 내용 입력할 문자열/배열 생성
15    ifstream file(league + ".txt");
16    string bufferfirstName;
17    string bufferlastName;
18    string bufferTeam;
19    int bufferStats[STAT_SIZE] = {};
20
21    while (file >> bufferfirstName >> bufferlastName >> bufferTeam) {
22        // 데이터 차례대로 저장
23        for (int i = 0; i < STAT_SIZE; i++)
24            file >> bufferStats[i];
25
26        // 소속 리그에 맞추어 리스트에 삽입
27        if (league == "KBO") {
28            KBOPlayer* new_player = new KBOPlayer(bufferfirstName + " " + bufferlastName, bufferTeam, bufferStats);
29            list.insert_back(new_player);
30        }
31        else if (league == "MLB") {
32            MLBPlayer* new_player = new MLBPlayer(bufferfirstName + " " + bufferlastName, bufferTeam, bufferStats);
33            list.insert_back(new_player);
34        }
35    }
36
37    // 리스트 내 노드의 비율 스탯 초기화
38    list.calculate_av();
39    sort(mode, order);
40 }

```

Handler의 생성자는 league 문자열을 입력 받아 KBO/MLB 중 한 리그를 선택하도록 하며, 정렬 모드를 타율, 내림차순으로 초기화한다.

리스트를 초기화하는 initialize 함수의 경우 우선 delete_front 함수를 이용해 리스트 내 모든 노드를 비운 뒤, ifstream을 통해 KBO.txt나 MLB.txt 파일을 열어 파일을 순회하면서 데이터를 읽는다. 이 때 league에 따라 KBOPlayer나 MLBPlayer 중 하나를 선택하여 새 객체를 만들고 list에 삽입한다. 삽입이 완료되면 리스트 내 모든 노드의 비율 스탯을 계산한 뒤 sort 함수를 통해 타율, 내림차순으로 정렬한다.

```

81 // 리스트 내 플레이어 삭제
82 void Handler::del_player(string team, string name) {
83     Player* tmp = NULL;
84     while (list.search_list(tmp, team, name) && tmp != NULL) { // 팀/이름에 대해 검색
85         list.delete_node(tmp); // 발견 시 해당 노드 삭제
86     }
87     list.calculate_av(); // 리스트 내 스탯 다시 계산
88 }
89

```

del_player 함수의 경우 search_list 함수를 통하여 팀, 이름 조건에 맞는 모든 플레이어를 del_node 함수를 통해 삭제한다. 만약 team과 name이 공백일 경우 모두 삭제하며, team이나 name 중 하나만 공백인 경우 남은 다른 조건에 맞는 모든 플레이어를 삭제한다. 삭제 완료 후 calculate_av 함수를 통해 비율 스탯을 다시 계산하도록 한다.

```

90     // 힙을 이용해 리스트 정렬
91     void Handler::sort(int _mode, int _order) {
92         // 힙 초기화 (동적 할당, 모드/차순 설정)
93         int member = list.getnPlayer();
94         Heap heap(member*2);
95         mode = _mode; order = _order;
96         heap.setMode(mode);
97         heap.setOrder(order);
98
99         // list의 모든 데이터를 heap으로 보낸 뒤 다시 list에 삽입
100        for (int i = 0; i < member; i++)
101            heap.push(list.delete_front());
102
103        for (int i = 0; i < member; i++) {
104            list.insert_back(heap.pop());
105        }
106    }

```

sort 함수의 구현은 위와 같다. heap을 생성하고 입력 받은 mode와 order에 따라 heap을 설정한 뒤 list의 모든 데이터를 heap에 push한 후 다시 heap에서 pop하여 list에 삽입하면 정렬이 완료된다. 만약 mode = OPS, order = -1인 경우 OPS에 따라 오름차순으로 정렬된다.

```

..
42     // 리스트 내 데이터 출력
43     void Handler::print_stats(int count, string team, string name, double min, double max, bool reverse) {
44         // count가 입력되어 있지 않거나 리스트 내 데이터 수를 초과할 경우 모든 플레이어 출력
45         if (count == -1 || count > list.getnPlayer()) count = list.getnPlayer();
46         if (!list.isEmpty()) {
47             double head_stat = list.getHeadStat(mode);
48             double tail_stat = list.getTailStat(mode);
49             if (min == NO_RANGE) min = (order == 1) ? tail_stat : head_stat;
50             if (max == NO_RANGE) max = (order == 1) ? head_stat : tail_stat;
51         }
52         list.print_out(mode, count, team, name, min, max, reverse);
53         cout << endl;
54     }
55
56     // 리스트 내 데이터 출력: 팀/이름만 입력
57     void Handler::print_stats(string team, string name) {
58         double min = NO_RANGE, max = NO_RANGE;
59         if (!list.isEmpty()) {
60             double head_stat = list.getHeadStat(mode);
61             double tail_stat = list.getTailStat(mode);
62             min = (order == 1) ? tail_stat : head_stat;
63             max = (order == 1) ? head_stat : tail_stat;
64         }
65         list.print_out(mode, list.getnPlayer(), team, name, min, max, false);
66         cout << endl;
67     }
68
69     // 리스트 내 데이터 출력: 범위만 입력
70     void Handler::print_stats(double min, double max) {
71         if (!list.isEmpty()) {
72             double head_stat = list.getHeadStat(mode);
73             double tail_stat = list.getTailStat(mode);
74             if (min == NO_RANGE) min = (order == 1) ? tail_stat : head_stat;
75             if (max == NO_RANGE) max = (order == 1) ? head_stat : tail_stat;
76         }
77         list.print_out(mode, list.getnPlayer(), ALL, ALL, min, max, false);
78         cout << endl;
79     }

```

```

81 // 리스트 내 데이터 출력: 순서 뒤집기 여부만 입력
82 void Handler::print_stats(bool reverse) {
83     list.print_out(mode, list.getnPlayer(), ALL, ALL, NO_RANGE, NO_RANGE, reverse);
84     cout << endl;
85 }

```

리스트 내 데이터를 조건에 맞추어 출력하는 print_stats 함수는 위와 같이 오버로딩을 통해 구현되었다. 가장 위의 함수는 상위 n명, 이름, 팀, 최소/최대 범위, 순서 뒤집기 여부를 모두 입력 받으며, 밑 세 개 함수는 각각 팀/이름, 최소/최대 범위, 순서 뒤집기 여부만 입력 받도록 되어있다.

또한 Handler.h에 default 값이 지정되어 있기 때문에, 전체 플레이어를 내림차순으로 출력하고 싶으면 handler.print_stats();만 입력하면 된다. 만약 count가 출력할 수 있는 범위를 벗어나면 모든 플레이어를 출력하도록 하고, 만약 최소/최대 범위가 지정 되어있지 않을 경우 정렬 순서에 따라 최소/최대 범위를 모든 플레이어가 포함될 수 있도록 변경한다. 그 후 list의 print_out 함수를 통해 조건에 맞는 플레이어를 출력하도록 한다.

```

108 // 리스트 내 플레이어 스탯 업데이트
109 void Handler::update(string _team, string _name, int _mode, int stat) {
110     Player* prev = NULL;
111     if (!_team.empty() && !_name.empty()) { // 이름과 팀이 둘다 공백이 아닐 때
112         if (list.search_list(prev, _team, _name) && prev != NULL) { // 해당 이름과 팀을 가진 선수가 발견되었을 때
113             // 해당 선수의 스탯을 복사
114             Player* tmp = NULL;
115             int tmp_stats[STAT_SIZE] = {};
116             for (int i = 0; i < STAT_SIZE; i++)
117                 tmp_stats[i] = prev->getStats()[i];
118             int diff = stat - prev->getStats()[_mode]; // 입력 받은 새 스탯과 원래 스탯의 차이
119
120             switch (_mode) {
121                 case B_1: case B_2: case B_3: case B_4: // 1루타/2루타/3루타/홈런의 경우
122                     tmp_stats[H] += diff; // 안타, 타석, 타수, 스탯 증가
123                 case K: case DP: // 삼진/병살의 경우
124                     tmp_stats[AB] += diff; // 타석, 타수, 스탯 증가
125                 case BB: case HBP: case IBB: case SH: case SF: // 볼넷/사구/고의사구/번트/희비의 경우
126                     tmp_stats[PA] += diff; // 타석, 스탯 증가
127                 case R: case RBI: // 득점, 타점의 경우
128                     tmp_stats[_mode] += diff; // 해당 스탯 증가
129                     break;
130                 default: // 나머지 경우 해당 스탯만 증가하기 불가능
131                     cout << "해당 스탯은 업데이트 할 수 없습니다." << endl << endl;
132                     return;
133             }
134
135             // 새 노드 생성 후 기존 노드 삭제
136             if (league == "KBO") tmp = new KBOPlayer(prev->getName(), prev->getTeam(), tmp_stats);
137             else if (league == "MLB") tmp = new MLBPlayer(prev->getName(), prev->getTeam(), tmp_stats);
138             list.delete_node(prev);
139
140             // 새 노드 삽입 후 비율 스탯 재계산 및 정렬
141             list.insert_back(tmp);
142             list.calculate_av();
143             sort(mode, order);
144         }
145     }
146     else return;
147 }

```

마지막으로 특정 플레이어의 스탯을 변경하는 update 함수의 구현은 위와 같다. 플레이어가 특정 되어야 하므로 팀/이름 공백 여부와 실제 존재 여부를 먼저 확인한 뒤, 발견한 플레이어의 스탯을

모두 tmp_stats에 복사하고, switch 문을 통해 변경하고자 하는 스탯이 무엇이었는지에 따라 스탯을 조작해준다(주석 참고). 그 후 기존의 노드를 삭제하고 변경된 스탯 정보를 가진 새 노드를 삽입한 뒤 비율 스탯을 다시 계산하고 재정렬하는 방식으로 구현을 마쳤다.

서론에서 구현하고자 한 기능을 다시 점검해보면, 모든 기능을 구현했음을 확인할 수 있다.

1. txt 파일에 저장된 선수의 이름/팀/기록을 읽어 데이터 구조에 저장: Initialize 함수
2. 데이터 구조에 저장된 스탯을 이용해 각종 개인 기록 및 리그 평균 기록을 계산: calculate_ratio, calculate_lg_ratio, calculate_av 함수 등
3. 각 기록에 따라 오름차순/내림차순으로 정렬: heap을 이용한 sort 함수
4. 이름, 팀, 데이터 범위 등 특정 조건을 이용해 검색: search_list, print_stats 함수
5. 특정 플레이어의 삭제 및 스탯 변경 기능: del_player, update 함수

EXAMPLE CASE

모든 기능을 구현하였으므로, main.cpp를 통해 해당 기능을 직접 사용해보도록 하였다.

```
1 #include "Handler.h"
2
3 int main(void) {
4     // KBO 리그와 MLB 리그에 대해 객체 생성
5     Handler KBO_handler("KBO");
6     Handler MLB_handler("MLB");
7
8     // 각 Handler 초기화 (KBO.txt, MLB.txt)
9     KBO_handler.initialize();
10    MLB_handler.initialize();
11
12    // KBO: wRC+ 오름차순 / MLB: wRC+ 내림차순 정렬
13    KBO_handler.sort(wRCP, -1);
14    MLB_handler.sort(wRCP);
15
16    // KBO: 모든 선수 출력 / MLB: 상위 10명 출력
17    KBO_handler.print_stats();
18    MLB_handler.print_stats(10);
19 }
```

1. Handler 객체를 KBO, MLB에 대해 각각 생성한 뒤 initialize 함수를 통해 초기화해주었고, KBO의 경우 wRC+ 오름차순으로 모든 선수를, MLB의 경우 wRC+ 내림차순으로 상위 10명만 출력하도록 하였다.

KBO League (상위 30명)		0.291	0.367	0.448	0.815		
이름	팀	타율	출루율	장타율	OPS	OPS+	wRC+
Hae-min Park	Twins	0.259	0.314	0.330	0.644	62.9	64.1
Seong-woo Chang	Wiz	0.224	0.310	0.379	0.689	67.5	66.5
Chan-ho Park	Tigers	0.271	0.343	0.340	0.684	70.2	71.5
Seonghan Park	Landers	0.298	0.375	0.374	0.749	81.5	82.6
Daeho Lee	Giants	0.288	0.335	0.443	0.778	86.8	83.8
Jin-Hyuk Roh	Dinos	0.280	0.353	0.429	0.782	92.0	84.5
Chi-hong An	Giants	0.284	0.354	0.436	0.790	90.3	89.1
Jihoon Choi	Landers	0.304	0.361	0.426	0.788	89.1	89.5
Jun-woo Jeon	Giants	0.304	0.350	0.445	0.795	91.1	89.6
Eunwon Jung	Eagles	0.276	0.379	0.370	0.749	87.4	91.3
Kyoung-Min Hur	Bears	0.289	0.360	0.398	0.759	92.5	93.1
Hyeseong Kim	Heroes	0.318	0.373	0.403	0.776	95.8	95.9
Chang-Ki Hong	Twins	0.286	0.390	0.355	0.745	90.8	95.9
Donghui Han	Giants	0.307	0.359	0.458	0.817	96.3	96.2
Mike Tauchman	Eagles	0.289	0.366	0.430	0.795	97.3	97.5
Jae-Hwan Kim	Bears	0.248	0.340	0.460	0.800	101.3	100.4
Yooseom Han	Landers	0.264	0.372	0.478	0.851	103.0	102.8
Nick Martini	Dinos	0.300	0.368	0.465	0.833	104.1	103.2
Socrates Brito	Tigers	0.311	0.354	0.494	0.848	107.8	106.3
Ji-hwan Oh	Twins	0.269	0.357	0.470	0.827	108.4	108.7
Eui-ji Yang	Dinos	0.283	0.380	0.480	0.860	110.9	109.5
Yasiel Puig	Heroes	0.277	0.367	0.474	0.841	110.7	111.7
Bo-Gyeong Moon	Twins	0.315	0.382	0.456	0.837	112.3	113.1
Hyun-Soo Kim	Twins	0.286	0.375	0.473	0.848	114.4	113.3
Jeong Choi	Landers	0.266	0.386	0.505	0.891	112.2	113.6
Kunwoo Park	Dinos	0.336	0.408	0.458	0.867	113.7	116.3
Byeong-Ho Park	Wiz	0.275	0.349	0.559	0.909	117.2	116.6
Sung-bum Na	Tigers	0.320	0.402	0.508	0.910	124.2	124.5
Jose Pirela	Lions	0.342	0.411	0.565	0.976	131.5	132.4
Junghoo Lee	Heroes	0.349	0.421	0.580	1.002	150.9	148.7

MLB League (상위 10명)		0.266	0.352	0.519	0.871		
이름	팀	타율	출루율	장타율	OPS	OPS+	wRC+
Aaron Judge	Yankees	0.311	0.439	0.686	1.125	156.7	154.5
Yordan Alvarez	Astros	0.306	0.415	0.613	1.027	135.8	134.6
P. Goldschmidt	Cards	0.317	0.402	0.578	0.980	125.5	125.4
Manny Machado	Padres	0.298	0.376	0.531	0.907	109.2	110.6
Ohtani Shohei	Angels	0.273	0.366	0.519	0.885	103.9	104.2
Pete Alonso	Mets	0.271	0.360	0.518	0.878	102.0	101.1
Mookie Betts	Dodgers	0.269	0.333	0.533	0.867	97.4	97.0
Auston Riley	Braves	0.273	0.336	0.528	0.864	97.1	94.5
Kyle Schwarber	Phillies	0.218	0.323	0.504	0.827	88.8	89.9
V.Guerrero Jr.	B.Jays	0.274	0.340	0.480	0.820	89.1	89.3

```
// 재초기화 후 출루율 내림차순 정렬, 리스트에 저장된 모든 선수 중 팀이 히어로즈인 선수 출력(뒤집어서)
KBO_handler.initialize();
KBO_handler.sort(OBA, 1);
KBO_handler.print_stats(1000, "Heroes", ALL, NO_RANGE, NO_RANGE, true);
```

2. KBO_handler를 초기화한 뒤, 이번엔 출루율을 기준으로 내림차순으로 정렬한 뒤, 팀이 히어로즈인 선수를 순서를 뒤집어서 출력하도록 하였다. 결과는 다음과 같다.

KBO League (상위 30명)		0.291	0.367	0.448	0.815		
이름	팀	타율	출루율	장타율	OPS	OPS+	wRC+
Yasiel Puig	Heroes	0.277	0.367	0.474	0.841	110.7	111.7
Hyeseong Kim	Heroes	0.318	0.373	0.403	0.776	95.8	95.9
Junghoo Lee	Heroes	0.349	0.421	0.580	1.002	150.9	148.7

4

```
// 재초기화 후 wRC+ 내림차순 정렬, wRC+가 100.0~140.0인 모든 선수 출력
KBO_handler.initialize();
KBO_handler.sort(wRC+);
KBO_handler.print_stats(100.0, 140.0);
```

3. KBO_handler를 다시 초기화한 뒤, wRC+를 기준으로 내림차순으로 정렬하고, wRC+가 100.0~140.0 사이인 선수를 출력하도록 하였다. 결과는 다음과 같다.

KBO League (상위 30명)		0.291	0.367	0.448	0.815		
이름	팀	타율	출루율	장타율	OPS	OPS+	wRC+
Jose Pirela	Lions	0.342	0.411	0.565	0.976	131.5	132.4
Sung-bum Na	Tigers	0.320	0.402	0.508	0.910	124.2	124.5
Byeong-Ho Park	Wiz	0.275	0.349	0.559	0.909	117.2	116.6
Kunwoo Park	Dinos	0.336	0.408	0.458	0.867	113.7	116.3
Jeong Choi	Landers	0.266	0.386	0.505	0.891	112.2	113.6
Hyun-Soo Kim	Twins	0.286	0.375	0.473	0.848	114.4	113.3
Bo-Gyeong Moon	Twins	0.315	0.382	0.456	0.837	112.3	113.1
Yasiel Puig	Heroes	0.277	0.367	0.474	0.841	110.7	111.7
Eui-ji Yang	Dinos	0.283	0.380	0.480	0.860	110.9	109.5
Ji-hwan Oh	Twins	0.269	0.357	0.470	0.827	108.4	108.7
Socrates Brito	Tigers	0.311	0.354	0.494	0.848	107.8	106.3
Nick Martini	Dinos	0.300	0.368	0.465	0.833	104.1	103.2
Yooseom Han	Landers	0.264	0.372	0.478	0.851	103.0	102.8
Jae-Hwan Kim	Bears	0.248	0.340	0.460	0.800	101.3	100.4

```
// 특정 팀/특정 선수 삭제 기능
KBO_handler.del_player("Landers");
KBO_handler.del_player("Heroes", "Junghoo Lee");
KBO_handler.print_stats(NO_RANGE);
```

4. 팀이 Landers이거나, 팀이 Heroes이면서 이름이 Junghoo Lee인 선수를 삭제하도록 하였다. 결과는 다음과 같다. (총 5명 삭제)

KBO League (상위 25명)		0.290	0.363	0.443	0.806		
이름	팀	타율	출루율	장타율	OPS	OPS+	wRC+
Jose Pirela	Lions	0.342	0.411	0.565	0.976	133.9	135.0
Sung-bum Na	Tigers	0.320	0.402	0.508	0.910	126.6	127.0
Byeong-Ho Park	Wiz	0.275	0.349	0.559	0.909	119.5	119.0
Kunwoo Park	Dinos	0.336	0.408	0.458	0.867	115.9	118.7
Hyun-Soo Kim	Twins	0.286	0.375	0.473	0.848	116.8	115.6
Bo-Gyeong Moon	Twins	0.315	0.382	0.456	0.837	114.6	115.4
Yasiel Puig	Heroes	0.277	0.367	0.474	0.841	113.0	114.0
Eui-ji Yang	Dinos	0.283	0.380	0.480	0.860	113.2	111.9
Ji-hwan Oh	Twins	0.269	0.357	0.470	0.827	110.7	111.0
Socrates Brito	Tigers	0.311	0.354	0.494	0.848	110.0	108.6
Nick Martini	Dinos	0.300	0.368	0.465	0.833	106.3	105.5
Jae-Hwan Kim	Bears	0.248	0.340	0.460	0.800	103.5	102.6
Mike Tauchman	Eagles	0.289	0.366	0.430	0.795	99.4	99.8
Donghui Han	Giants	0.307	0.359	0.458	0.817	98.4	98.5
Chang-Ki Hong	Twins	0.286	0.390	0.355	0.745	92.9	98.1
Hyeseong Kim	Heroes	0.318	0.373	0.403	0.776	97.9	98.1
Kyoung-Min Hur	Bears	0.289	0.360	0.398	0.759	94.6	95.2
Eunwon Jung	Eagles	0.276	0.379	0.370	0.749	89.3	93.5
Jun-woo Jeon	Giants	0.304	0.350	0.445	0.795	93.1	91.8
Chi-hong An	Giants	0.284	0.354	0.436	0.790	92.2	91.3
Jin-Hyuk Roh	Dinos	0.280	0.353	0.429	0.782	94.1	86.7
Daeho Lee	Giants	0.288	0.335	0.443	0.778	88.8	86.0
Chan-ho Park	Tigers	0.271	0.343	0.340	0.684	72.0	73.5
Seong-Woo Chang	Wiz	0.224	0.310	0.379	0.689	69.2	68.5
Hae-min Park	Twins	0.259	0.314	0.330	0.644	64.6	66.0

```

35     // 특정 선수 스탯 업데이트 기능
36     MLB_handler.print_stats();
37     MLB_handler.update("Angels", "Ohtani Shohei", PA, 1500);
38     MLB_handler.update("Angels", "Ohtani Shohei", B_4, 36);
39     MLB_handler.print_stats();
40
41     return 0;
42 }

```

5. 마지막으로 Angels의 Ohtani Shohei 선수의 타석을 1500으로, 홈런을 36으로 수정하였다. 타석은 타석만 따로 수정할 수 없기 때문에 에러 메시지를 출력한다.

MLB League (상위 15명)		0.266	0.352	0.519	0.871		
이름	팀	타율	출루율	장타율	OPS	OPS+	wRC+
Aaron Judge	Yankees	0.311	0.439	0.686	1.125	156.7	154.5
Yordan Alvarez	Astros	0.306	0.415	0.613	1.027	135.8	134.6
P. Goldschmidt	Cards	0.317	0.402	0.578	0.980	125.5	125.4
Manny Machado	Padres	0.298	0.376	0.531	0.907	109.2	110.6
Ohtani Shohei	Angels	0.273	0.366	0.519	0.885	103.9	104.2
Pete Alonso	Mets	0.271	0.360	0.518	0.878	102.0	101.1
Mookie Betts	Dodgers	0.269	0.333	0.533	0.867	97.4	97.0
Auston Riley	Braves	0.273	0.336	0.528	0.864	97.1	94.5
Kyle Schwarber	Phillies	0.218	0.323	0.504	0.827	88.8	89.9
V.Guerrero Jr.	B.Jays	0.274	0.340	0.480	0.820	89.1	89.3
Matt Olson	Braves	0.240	0.329	0.477	0.806	85.2	86.2
C. Walker	D.backs	0.242	0.324	0.477	0.801	83.9	83.6
Corey Seager	Rangers	0.245	0.319	0.455	0.774	78.3	78.4
Rowdy Tellez	Brewers	0.219	0.312	0.461	0.773	77.3	78.3
A. Santander	Orioles	0.240	0.308	0.455	0.763	75.0	72.9

해당 스탯은 업데이트 할 수 없습니다.

MLB League (상위 15명)		0.266	0.352	0.520	0.872		
이름	팀	타율	출루율	장타율	OPS	OPS+	wRC+
Aaron Judge	Yankees	0.311	0.439	0.686	1.125	156.4	154.3
Yordan Alvarez	Astros	0.306	0.415	0.613	1.027	135.6	134.4
P. Goldschmidt	Cards	0.317	0.402	0.578	0.980	125.3	125.2
Manny Machado	Padres	0.298	0.376	0.531	0.907	109.0	110.4
Ohtani Shohei	Angels	0.276	0.368	0.531	0.899	106.5	106.8
Pete Alonso	Mets	0.271	0.360	0.518	0.878	101.8	100.9
Mookie Betts	Dodgers	0.269	0.333	0.533	0.867	97.2	96.8
Auston Riley	Braves	0.273	0.336	0.528	0.864	96.9	94.4
Kyle Schwarber	Phillies	0.218	0.323	0.504	0.827	88.6	89.7
V.Guerrero Jr.	B.Jays	0.274	0.340	0.480	0.820	88.9	89.1
Matt Olson	Braves	0.240	0.329	0.477	0.806	85.0	86.0
C. Walker	D.backs	0.242	0.324	0.477	0.801	83.7	83.4
Corey Seager	Rangers	0.245	0.319	0.455	0.774	78.2	78.2
Rowdy Tellez	Brewers	0.219	0.312	0.461	0.773	77.2	78.1
A. Santander	Orioles	0.240	0.308	0.455	0.763	74.9	72.7

팀 스탯과 Ohtani Shohei의 개인 스탯, 그리고 모든 플레이어의 OPS+와 wRC+가 달라진 것을 확인할 수 있다.

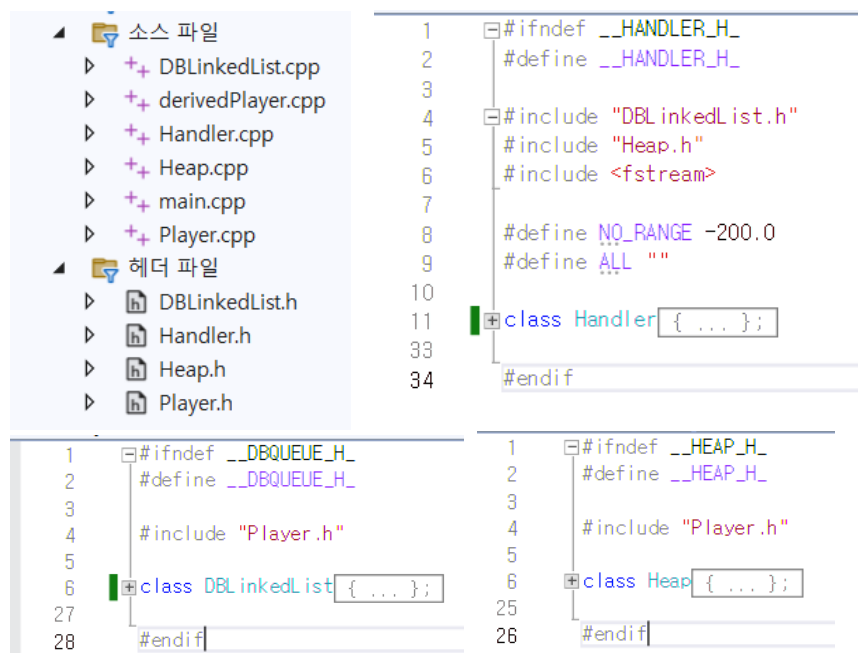
CONCLUSION

파일 입력과 이중 연결 리스트를 이용한 데이터 저장과 힙을 활용한 정렬, 상속과 다형성, 함수 오버로딩, 사용자 인터페이스 구현 등 C++의 여러 가지 기능과 자료 구조를 활용하여 가지고 있는 데이터를 다양하게 활용할 수 있도록 구현할 수 있었다.

이중 연결 리스트의 경우 삽입과 특정 데이터에 대한 삭제 기능에 대한 구현이 다른 자료 구조보다 쉽다는 큰 강점이 있었다. 또한 해당 프로젝트가 정렬 및 조건에 따라 상위/하위 n개의 데이터만 추출해도 되는 상황이 많기 때문에 head나 tail부터 시작하여 next/prev를 통해 노드를 순회하는 것으로 출력을 구현할 수 있어 프로젝트 상황에 아주 적합하였다.

이중 연결 리스트의 단점으로는 배열 등과 달리 index를 통해 직접적으로 특정 노드에 접근할 수 없고, 두 노드 사이의 위치를 바꾸는 기능을 구현하는 것이 까다롭기 때문에 정렬의 구현이 어려우며 또한 정렬 시간 역시 오래 걸린다는 것이 있다. 이는 또다른 자료 구조인 힙을 통해 보완하였다. 연결 리스트의 모든 노드 삭제 및 삽입, 힙의 모든 노드 삽입 및 삭제 시간이 추가적으로 들지만, 정렬 기능의 구현이 아주 쉽고 효율성 또한 높기 때문에 연결 리스트의 단점을 보완해줄 수 있다. 힙 트리의 장점을 극대화하여 상위 n개의 데이터만을 리스트에 저장하는 방식으로 구현하면 힙 데이터 삭제 후 리스트 삽입에 드는 시간 동안 크게 줄일 수 있을 것이다. 이러한 장점은 다루는 선수의 데이터가 증가할수록 극대화될 것이다.

따라서 두 자료 구조의 활용은 위 프로젝트와 같은 문제 상황에 대해 아주 적합하다고 결론 지을 수 있다.



The image shows a screenshot of a C++ project in an IDE. On the left, a file explorer displays the project structure:

- 소스 파일 (Source Files)
 - DBLinkedList.cpp
 - derivedPlayer.cpp
 - Handler.cpp
 - Heap.cpp
 - main.cpp
 - Player.cpp
- 헤더 파일 (Header Files)
 - DBLinkedList.h
 - Handler.h
 - Heap.h
 - Player.h

On the right, three code snippets are shown:

```
1  #ifndef __HANDLER_H_
2  #define __HANDLER_H_
3
4  #include "DBLinkedList.h"
5  #include "Heap.h"
6  #include <fstream>
7
8  #define NO_RANGE -200.0
9  #define ALL ""
10
11 class Handler { ... };
33
34 #endif
```

```
1  #ifndef __DBQUEUE_H_
2  #define __DBQUEUE_H_
3
4  #include "Player.h"
5
6  class DBLinkedList { ... };
27
28 #endif
```

```
1  #ifndef __HEAP_H_
2  #define __HEAP_H_
3
4  #include "Player.h"
5
6  class Heap { ... };
25
26 #endif
```

```

1  #ifndef __PLAYER_H_
2  #define __PLAYER_H_
3
4  #include <iostream>
5  using namespace std;
6
7  enum STAT { ... };
16
17  enum RATIO_STAT { ... };
29
30  #define STAT_SIZE 16 // 기본 스탯의 개수
31  #define RATIO_STAT_SIZE 8 // 비율 스탯의 개수
32
33  class Player { ... };
69
70  /* ... */
83
84  class KBOPlayer { ... };
97
98  class MLBPlayer { ... };
110
111  // 두 플레이어의 특정 스탯을 비교
112  int compStat(Player*, Player*, int);
113
114  #endif

```

각 헤더 파일의 경우 private, protected, public 접근 제한자를 활용하여 사용자가 private, protected 데이터에 직접 접근할 수 없도록 캡슐화를 적절히 구현하였다. 각 private 멤버에 접근할 때는 getter와 setter 등을 이용해 외부에서 직접 변경하지 못하도록 하였다. Player 클래스의 상속은 IS-A 관계를 유지하도록 하였고, 순수 가상 함수를 이용하여 다형성을 통해 각 리그에 대해 각각 호환되는 데이터구조를 만들지 않고 모든 데이터를 Player*로 처리하도록 구현하였다.

이 프로젝트는 위와 같이 네 개의 헤더 파일과 여섯 개의 소스 파일로 구성하였는데, Player.h는 노드에 대한 정보와 기능, DBLinkedList.h는 이중 연결 리스트의 기능, Heap.h은 힙의 기능 만을 구현하여 최종적으로 이를 활용해 데이터를 처리하는 것은 Handler 객체만이 할 수 있도록 파일 구조를 구축하였다. 따라서 main.cpp에서는 최종적으로 Handler.h 만을 이용하여 모든 기능을 사용할 수 있도록 하였다. 이를 통해 각 헤더의 독립성 유지, 각 객체의 기능에 대한 변경 및 유지보수가 쉽도록 하였다.

그 외에 스탯의 수나 스탯의 이름 등 여러 가지 상수를 열거형이나 매크로 상수로 지정하여 후에 변경이 쉽도록 하였으며, 상속과 부모 클래스의 포인터를 이용해 데이터를 다루게 함으로써 앞으로 다른 리그를 추가하거나 변경하도록 할 때 많은 수정을 하지 않도록 코드를 구현하였다.

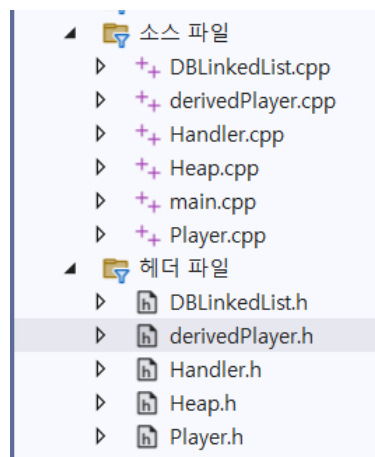
위 프로그램을 발전시켜 스탯 변경이나 정렬에 대한 결과를 ofstream을 통하여 저장하거나, 사용자가 프로그램 내부에서 직접 여러 스탯을 변경, 검색하면서 여러 결과를 관찰할 수 있도록 인터페이스를 구현하는 등 많은 변형을 통해 분석에 더욱 활용할 수 있도록 만들 수 있을 것이다.

ADDITIONAL UTILIZATION - KBL

해당 프로젝트를 기반으로 발표를 한 결과, 해당 프로젝트가 유지 보수의 용이함을 위해 객체지향 프로그래밍의 특징이자 강점인 상속, 다형성과 캡슐화 등 다양한 기능을 활용한 것은 확인되지만, 이를 실제로 이용한 예시가 부족하다는 점에 대한 피드백을 받았다. 이를 해결하기 위해 야구가 아닌 축구와 같은 타 종목의 리그에 대해서 구현해보는 것을 고려해보았다.

따라서 기존 프로젝트를 이용하여 한국프로농구리그(KBL)의 여러 가지 스탯 데이터를 활용하여 같은 방법으로 데이터베이스 정렬/검색 기능을 추가해보기로 하였다. 앞서 유지 보수가 용이하도록 다양한 매크로 상수, 캡슐화, 상속, 가상 함수, Handler 인터페이스 등 여러 가지 기법을 사용하였기 때문에, 이를 활용하여 **코드를 최대한 변경하지 않고** 추가해보기로 하였다.

우선 다음과 같이 KBO, MLB, 그리고 KBL 리그의 상수나 클래스 정보 등을 derivedPlayer 헤더 파일을 추가하여 분리하도록 하였다. 따라서 STAT, RATIO_STAT 과 같은 열거형 정보 등은 전부 derivedPlayer.h 로 이동하였다.



다음으로, 기존 프로젝트가 타 스포츠가 아닌 '야구' 리그에 대한 정보만 추가될 것이라고 생각하고 코드를 구현하였기 때문에, Player 클래스를 수정하기로 하였다.

```
7  class Player {                                // 선수의 정보를 관리하는 클래스
8      protected:
9          // 이름, 팀, 스탯 수치를 저장할 배열
10         string name;
11         string team;
12         int* stats;
13         double* ratio_stats;
14
15         // 이중 연결 리스트에서 다음 노드, 이전 노드
16         Player* next;
17         Player* prev;
```

우선 기존의 stats와 ratio_stats는 각 스포츠가 요구하는 스탯의 수에 따라 그 사이즈가 변경되도록 배열이 아닌 포인터로 변경하였다. 또한 각 스포츠마다 스탯의 계산 방법이 달라지는 것은 필연적이기 때문에 자식 클래스로 이동하였다.

```

35
36     virtual void print_stats(void) = 0; // Player의 스탯 출력
37     virtual void print_league(int) = 0; // 해당 Player가 속한 리그의 평균 스탯 출력
38     virtual void calculate_lg_ratio(void) = 0; // 리그 평균 스탯 기반 플레이어 비율 스탯 계산
39 };

```

또한 각 종목마다 스탯의 출력 방식 또한 달라지기 때문에 print_stats 함수 또한 순수 가상 함수로 선언하여 각 자식 클래스에서 구현하도록 하였다.

```

1  #ifndef __DR_PLAYER_H_
2  #define __DR_PLAYER_H_
3
4  #include "Player.h"
5
6  /* ... */
19
20  enum BASEBALL_STAT { ... };
29
30  enum BASEBALL_RATIO_STAT { ... };
42
43  #define BASEBALL_STATS_SIZE 16 // 기본 스탯의 개수
44  #define BASEBALL_RATIO_SIZE 8 // 비율 스탯의 개수
45
46  class KBOPlayer { ... };
63
64  class MLBPlayer { ... };
80
81  /* ===== */
82
83  enum BASKETBALL_STAT { ... };
92
93  enum BASKETBALL_RATIO_STAT { ... };
101
102  #define BASKET_STATS_SIZE 16
103  #define BASKET_RATIO_SIZE 6
104
105  class KBLPlayer { ... };
121

```

derivedPlayer.h의 구성은 위와 같이 각 스탯의 enum, 각 스탯의 수를 나타내는 매크로 상수, 그리고 각 자식 클래스의 정보를 표시하도록 하였다. 또한 농구의 스탯을 나타내는 BASKETBALL_STAT과 BASKETBALL_RATIO_STAT을 따로 작성하였다.

KBO와 MLB 자식 클래스는 기존과 동일하지만 calculate_ratio 함수와 print_stats 함수를 자식 클래스에서 구현하도록 하였다. KBL 클래스는 MLB 클래스와 구성이 동일하다.

농구 리그는 출전 시간, 2점슛과 시도, 3점슛과 시도, 필드골과 시도, 자유투와 시도, 공격/수비 리바운드, 어시스트, 실책, 스틸, 블록슛, 득점 등 총 16개를 기본 스탯으로 하고, 2점슛 성공률,

3 점슛 성공률, 필드골 성공률, 자유투 성공률, 유효슈팅 성공률과 야구 리그에서 wRC+와 역할이 동일한 KBL Efficiency 를 비율 스탯으로 하도록 하였다.

```

105 class KBLPlayer : public Player {
106     private:
107         static int nlgPlayer;
108         static int lg_stats[BASKET_STATS_SIZE];
109         static double lg_ratio_stats[BASKET_RATIO_SIZE];
110
111         // 기본 스탯을 기반으로 비율 스탯을 계산
112         void calculate_ratio(int*, double*);
113
114     public:
115         KBLPlayer(string, string, int[BASKET_STATS_SIZE]);
116         ~KBLPlayer();
117         void print_stats(void); // Player의 스탯 출력
118         void print_league(int); // 해당 Player가 속한 리그의 평균 스탯 출력
119         void calculate_lg_ratio(void); // 리그 평균 스탯 기반 플레이어 비율 스탯 계산
120 };

164 /* ===== */
165
166 int KBLPlayer::nlgPlayer = 0;
167 int KBLPlayer::lg_stats[BASKET_STATS_SIZE] = {};
168 double KBLPlayer::lg_ratio_stats[BASKET_RATIO_SIZE] = {};
169
170 // KLB Player 클래스의 생성자
171 KBLPlayer::KBLPlayer(string _name, string _team, int _stats[BASKET_STATS_SIZE]) { ... }
172
173 // KBL Player 클래스의 소멸자
174 KBLPlayer::~KBLPlayer() { ... }
175
176 // Player의 스탯 출력
177 void KBLPlayer::print_stats(void) { ... }
178
179 // format에 맞추어 KBL 리그의 평균 비율 스탯 출력
180 void KBLPlayer::print_league(int count) {
181     cout.precision(3); cout << fixed;
182     cout << "KBL League (상위 " << count << "명)" << lg_ratio_stats[PR_2] << "%%" << lg_ratio_stats[PR_3]
183     << "%%" << lg_ratio_stats[FGR] << "%%" << lg_ratio_stats[FTR] << "%%" << lg_ratio_stats[TSR] << endl << endl;
184     cout << "이름%%팀%%2점슛%%3점슛%%야투%%자유투%%유효슛%%EFF" << endl;
185     cout << "-----" << endl;
186 }
187
188 // 기본 스탯을 기반으로 비율 스탯을 계산
189 void KBLPlayer::calculate_ratio(int* _stats, double* _ratio) { ... }
190
191 // 리그 평균 스탯 기반 플레이어 비율 스탯 계산
192 void KBLPlayer::calculate_lg_ratio(void) { ... }

```

KBO, MLB, KBL 자식 클래스의 구성은 모두 동일하므로 KBL 을 예시로 들면 위와 같다. 자식 클래스로 옮겨진 print_stats 함수와 calculate_ratio 함수의 구현 또한 동일하다. 기존 프로젝트와 차이점으로는 기존의 DBLinkedList 의 print_out 함수에서 출력하던 스탯 레이아웃(213 줄)을 클래스의 print_league 함수가 출력하도록 한 것과, 생성자와 소멸자에서 각 종목의 스탯 수에 따라 동적 할당하고 해제하는 코드를 추가한 것 뿐이다.

이중 연결 리스트(DBLinkedList) 클래스의 경우 앞서 언급한 print_out 함수의 출력 방식 변경을 제외하고 전혀 변경점이 없으며, 이는 힙(Heap) 클래스 또한 마찬가지이다. 두 데이터 구조 모두 자식 클래스에 상관 없이 Player*로 모든 작업을 처리하도록 다형성을 구현하였기 때문이다. 정렬 기준 또한 compStat 함수를 이용해 어떠한 종목이나 비율 스탯이든 힙 정렬을 사용할 수 있도록 구현되었다.

```

12  class Handler {           // 유저 인터페이스
13      private:
14          DBLinkedList list; // 데이터 저장 연결 리스트
15          string league;    // Player가 속한 리그
16          int mode;         // 현재 정렬 중인 스탯
17          int order;        // 오름차순/내림차순 여부
18          int stat_size;    // 종목에 따라 스탯의 수 결정

```

유저와 실제로 상호 작용하는 Handler 클래스에는 위와 같이 league 에 따라 스탯의 수를 결정하는 stat_size 라는 멤버를 추가해주었다. 따라서 기존의 STAT_SIZE 상수를 사용하던 모든 부분을 stat_size 멤버 변수로 변경해주었다.

```

3      // Handler의 생성자, league 문자열 초기화
4      Handler::Handler(string _league) : league(_league) {
5          mode = 0; order = 1;
6          stat_size = (league == "KBL") ? BASKET_STATS_SIZE : BASEBALL_STATS_SIZE;
7      }

```

그 후 Handler 의 생성자에서 리그에 따라 stat_size 멤버 변수가 결정되도록 하였다.

```

127         Player* tmp = NULL;           // 해당 선수의 스탯을 복사
128         int *tmp_stats = new int[stat_size];
129         for (int i = 0; i < stat_size; i++)
130             tmp_stats[i] = prev->getStats()[i];
131         int diff = stat - prev->getStats()[_mode];           // 입력 받은 새 스탯과 원래 스탯의 차이
132
133         if (league == "KBO" || league == "MLB") { ... }
134         else if (league == "KBL") { ... }
135
136         // 새 노드 생성 후 기존 노드 삭제
137         if (league == "KBO")     tmp = new KBOPlayer(prev->getName(), prev->getTeam(), tmp_stats);
138         else if (league == "MLB") tmp = new MLBPlayer(prev->getName(), prev->getTeam(), tmp_stats);
139         else if (league == "KBL") tmp = new KBLPlayer(prev->getName(), prev->getTeam(), tmp_stats);
140         list.delete_node(prev); delete[] tmp_stats;

```

또한, initialize 와 update 함수에서 새 Player 를 생성하는 코드를 KBL 리그에 맞추어 하나 생성해주었다. update 함수에서 스탯을 변경하기 위해 생성되는 tmp_stats 는 기존의 배열에서 종목에 따라 동적 할당 하도록 포인터로 구현하였으며, 기존의 switch 문을 KBO/MLB 와 KBL 기준으로 나누어 스탯을 적절히 수정하도록 코드를 추가해주었다. sort, del_player, update, print_stats 등의 함수는 전혀 수정하지 않아도 되었다.

이것으로 구현이 완료되었다. 데이터 구조인 이중 연결 리스트나 Heap 의 메커니즘은 전혀 변경하지 않고, Player 클래스의 구조 변경 및 새 자식 클래스 추가와 Handler 클래스의 생성자, initialize, update 함수의 수정으로 아예 새 종목에 대한 데이터베이스를 구현한 것이다.

```

43 // KBL handler
44 Handler KBL_handler("KBL"); // KBL 리그 Handler 생성
45 KBL_handler.initialize(); // 초기화 (정렬 기준: EFF, 내림차순)
46 KBL_handler.print_stats(); // 출력
47 KBL_handler.sort(TSR, -1); // 유효 슈팅 비율에 대해 정렬 (오름차순)
48 KBL_handler.print_stats("Egis", ALL); // Egis 소속 선수만 출력
49 KBL_handler.print_stats(0.45, 0.5); // 유효 슈팅 비율이 0.45~0.5 사이인 선수만 출력
50 KBL_handler.del_player("Egis"); // Egis 소속 선수 모두 삭제
51 KBL_handler.update("Phoebus", "Shawn Long", MIN, 20000); // Shawn의 출전 시간 20000으로 변경
52 KBL_handler.sort(EFF); // EFF에 대해 정렬 (오름차순)
53 KBL_handler.print_stats(); // 출력

```

다음은 사용 예시이다. main 함수의 다음과 같은 코드를 입력해주었다.

KBL League (상위 10명)							
이름	팀	2점슛	3점슛	야투	자유투	유효슛	EFF
Shawn Long	Phoebus	0.549	0.372	0.533	0.797	0.451	1819.2
Dedric Lawson	Orions	0.534	0.361	0.487	0.752	0.433	1472.7
Isaiah Hicks	Thunder	0.572	0.355	0.533	0.751	0.468	1451.0
Jameel Warney	Knights	0.494	0.329	0.464	0.702	0.399	1409.6
Guuna Ra	Egis	0.610	0.214	0.599	0.897	0.490	1396.2
G.chang Song	Egis	0.564	0.335	0.504	0.602	0.417	1377.3
Justin Knox	Phoebus	0.596	0.371	0.568	0.700	0.467	1209.2
Tyler Davis	Egis	0.589	0.136	0.567	0.847	0.472	1186.2
L. Williams	KGC	0.637	0.364	0.623	0.702	0.499	1125.3
Brandon Brown	SonicBm	0.570	0.333	0.511	0.750	0.447	1104.5

KBL League (상위 10명)							
이름	팀	2점슛	3점슛	야투	자유투	유효슛	EFF
G.chang Song	Egis	0.564	0.335	0.504	0.602	0.417	1377.3
Tyler Davis	Egis	0.589	0.136	0.567	0.847	0.472	1186.2
Guuna Ra	Egis	0.610	0.214	0.599	0.897	0.490	1396.2

KBL League (상위 10명)							
이름	팀	2점슛	3점슛	야투	자유투	유효슛	EFF
Shawn Long	Phoebus	0.549	0.372	0.533	0.797	0.451	1819.2
Justin Knox	Phoebus	0.596	0.371	0.568	0.700	0.467	1209.2
Isaiah Hicks	Thunder	0.572	0.355	0.533	0.751	0.468	1451.0
Tyler Davis	Egis	0.589	0.136	0.567	0.847	0.472	1186.2
Guuna Ra	Egis	0.610	0.214	0.599	0.897	0.490	1396.2
L. Williams	KGC	0.637	0.364	0.623	0.702	0.499	1125.3

KBL League (상위 7명)							
이름	팀	2점슛	3점슛	야투	자유투	유효슛	EFF
Shawn Long	Phoebus	0.549	0.372	0.533	0.797	0.451	6454.4
Dedric Lawson	Orions	0.534	0.361	0.487	0.752	0.433	1472.7
Isaiah Hicks	Thunder	0.572	0.355	0.533	0.751	0.468	1451.0
Jameel Warney	Knights	0.494	0.329	0.464	0.702	0.399	1409.6
Justin Knox	Phoebus	0.596	0.371	0.568	0.700	0.467	1209.2
L. Williams	KGC	0.637	0.364	0.623	0.702	0.499	1125.3
Brandon Brown	SonicBm	0.570	0.333	0.511	0.750	0.447	1104.5

그 결과는 위와 같다. 야구 리그에 대해서만 구현했던 모든 기능(정렬, 차순, 검색, 범위 지정, 삭제, 수정)이 코드 수정 없이 정상적으로 잘 작동되는 것을 확인할 수 있다.

CONCLUSION

최종적으로 위 프로젝트에서 새 종목의 리그를 추가하기 위해 수정한 부분은 다음과 같다.

- Player 클래스의 구조 변경 및 derivedPlayer.h 파일 생성
 - Player 클래스의 stats, ratio_stats 동적 할당/해제 하도록 변경
 - Player 클래스의 calculate_ratio, print_stats 함수 자식 클래스로 이동
- 자식 클래스 수정 및 KBL 클래스 추가
 - calculate_ratio, print_stats 를 각 자식 클래스에 추가하고, stats 포인터 동적 할당
 - KBL 리그에서 사용하는 STAT, RATIO_STAT, SIZE 매크로 상수 추가
 - KBL 클래스에 대해 각종 함수 구현 (calculate_ratio 등)
- Handler 클래스 및 자료 구조 변경
 - DBLinkedList – print_out 함수 출력 방식 변경 (print_league 가 레이아웃 출력)
 - Handler 의 stat_size 변수 추가. 생성자에서 지정. 기존 STAT_SIZE 를 사용하던 initialize 와 update 함수 변경.
 - initailize 에 KBL Player 생성 코드 추가
 - update 의 tmp_stats 동적 할당으로 변경. KBL 스탯에 대한 switch 문 추가, KBL Player 생성 코드 추가.

또한, Player 클래스의 구조 변경이나 동적 할당, stat_size 변수 추가 등은 기존의 야구 리그만을 반영했던 코드에서 모든 종목에 대해 구현이 가능하도록 변경한 것이기 때문에, 다음에 야구, 농구 뿐만 아닌 축구나 테니스 등의 DB 를 구축할 때는 1. 자식 클래스 + 상수 추가 2. Handler 에서 생성 코드 추가 3. update switch 문 변경만 해주어도 된다.

이중 연결 리스트와 힙은 종목, 리그, 스탯의 추가와 상관 없이 변경을 하지 않아도 정상적으로 모든 동작을 수행할 수 있으며, Handler 또한 종목에 따라 반드시 변경되어야 할 초기화나 플레이어 생성을 제외한 정렬, 삭제, 출력 기능은 정상적으로 작동하는 점을 확인할 수 있었다.

이를 통해 기존 결론에서 주장했던 각 자료 구조의 독립성 유지와 캡슐화, 다형성 구현이 모두 적절히 이루어졌고, 따라서 유지 보수가 용이하도록 프로젝트가 구현됐음을 다시 한번 확인할 수 있었다.